

National University of Singapore
School of Computing

Semester 1, AY2026-27

CS4246/CS5446

Reinforcement Learning and Sequential Decision Making

Issued: 2 September 2026

Due: Friday 18 September 2026 23:59

Rush Hour Adventures: Episode I - AI Planning for the Elevator Challenge

Guidelines

Please complete the Assignment in your existing Tutorial Teams as formed on Canvas. **ONE person from each team** should submit ALL the answers, and grading will be done team-wise. The assignment is graded out of **10 points** in total.

On collaboration

You are encouraged to discuss solution ideas. However, each team *must write up the solutions independently*. It is considered plagiarism if the solution write-up is highly similar to other teams' write-ups or other sources.

Generative AI Usage & Declaration

The use of Generative AI tools (such as ChatGPT, Claude, Copilot, etc.) is permitted for this assignment in accordance with course guidelines.

- Declaring AI usage is not graded (AI usage records, prompts, and audit notes will not impact your score).
- If you wish to document or declare your AI tool usage, prompt logs, or reflections for transparency, you may do so by filling out `ai_usage.md`.
- Students remain solely responsible for the technical correctness and understanding of all submitted code and models.

Assignment Solutions Submission

- You can download the programming package from Canvas or from [this GitHub repo](#).
- No late submissions are allowed.
- Please write the following on the top of the solution files:
 - *Name* and *matric number* (starting with “A...”) of **all team members** as they appear on Canvas.
 - Collaborators (write **None** if no collaborators)
 - Sources, if any, or if you obtained the solutions through research, e.g., through the web.

- The coding package should be zipped and named:

`netid1-netid2.zip` (or `netid1-netid2-netid3.zip` if 3-person team),

where `netid` is the id starting with “E...”. **Only submit the zipped file to Canvas.** The folder structure should be strictly formed as follows:

```
netid1-netid2.zip (or netid1-netid2-netid3.zip)
├── EleEnv
├── 1_PDDL.ipynb
├── 2_hierarchical.ipynb
├── 1_PDDL_solution.py
├── 2_hierarchical_solution.py
├── 3_freeform_planning.ipynb
├── 3_freeform_planning_solution.py
└── aiusage.md
```

- **IMPORTANT!** We are grading based on the three python scripts, you are going to code in the Jupyter notebooks, after completion, remember to **COPY or IMPLEMENT the solutions to the corresponding python scripts**, detailed instructions as follows.

Programming

Welcome to the Elevator Challenge! This assignment will immerse you in a classic AI planning problem set in the bustling environment of a high-rise building during the evening rush hour. Your mission, should you choose to accept it, is to design and implement planning solutions for a single elevator serving passengers with different start and destination floors. This seemingly simple scenario brings a host of planning challenges involving state modelling, capacity constraints, ordering, and service quality.

Background: The Elevator Domain

The Elevator Domain is a well-known problem in AI planning that captures the intricacies of coordinating movements in a constrained environment. Imagine a busy office building with several floors and passengers travelling between different parts of the building. There is **ONLY ONE** elevator, but its capacity is specified by each problem configuration. The elevator can start from any floor and may need to make several trips to serve all passengers while respecting its door and occupancy constraints.

For more insight into the elevator’s operation, please explore the provided interactive Jupyter notebook, `0_env_walkthrough.ipynb`, where you can play around with different scenarios and understand the domain constraints and dynamics in detail.

About the Unified Planning Library

This assignment is part of a broader exploration of AI planning techniques, leveraging tools from the [AIPlan4EU](#) project, a European initiative aimed at advancing AI planning technologies. AIPlan4EU provides a [Unified Planning Library](#) designed to offer flexible and powerful planning solutions, integrating different planning paradigms, such as classical PDDL (Planning Domain Definition Language) and Hierarchical Task Network (HTN) planning.

You will also use the [Aries interface](#), a user-friendly platform that makes it easy to interact with these planners and visualize your solutions. These tools will help you tackle the elevator planning problem effectively.

Instructions on Solution

Problems 1 and 2 can be worked on in the Jupyter notebooks `1_PDDL.ipynb` and `2_hierarchical.ipynb`. An optional bonus problem is provided in `3_freeform_planning.ipynb`.

ONLY the python scripts will be graded, i.e., `1_PDDL_solution.py`, `2_hierarchical_solution.py`, and, if attempted, `3_freeform_planning_solution.py`. So after completion on Jupyter Notebooks, please copy or implement the required blocks to the corresponding python scripts. All blocks you should copy are indicated by comments like (where N indicates different blocks):

```
# COPY-FLAG-N-START
# COPY-FLAG-N-END
```

Problem 1: Classical Planning (2 points)

In the first task, you will implement a solution using the Planning Domain Definition Language (PDDL), a standard language for expressing planning problems. You will first model passenger requests with arbitrary start and goal floors, then extend the model with destinations, completed trips, and a configurable elevator capacity. The model and problem generators should work for every valid configuration described in the notebook.

Instructions: Complete this part in the `1_PDDL.ipynb` Jupyter notebook, following the detailed guidelines provided. After completion, copy the required blocks to `1_PDDL_solution.py`. There are four parts to the problem: foundational domain, foundational problem generator, capacity-aware domain, and capacity-aware problem generator. Only the two capacity-aware tasks are graded (1 point each, 2 points total). The foundational tasks are ungraded. AI assistance is permitted and encouraged, but you are responsible for checking that the resulting model captures the intended rules. The solution script includes an optional space for concise AI-audit notes; it is not read by the autograder, and carries no marks. Prompt transcripts and AI usage records are not graded.

Problem 2: Hierarchical Planning (8 points)

Hierarchical Task Network (HTN) planning is a method that breaks down complex tasks into simpler sub-tasks. In the second task, you will carry the passenger destination and capacity representation from Problem 1 into a hierarchical model, specify primitive actions and decomposition methods, construct an ordered task network, and design a service-order policy for passengers with deadlines and time-dependent utility.

Instructions: Work through this part in the `2_hierarchical.ipynb` Jupyter notebook, where step-by-step instructions will guide you in implementing and testing your hierarchical planning approach. You will directly implement your code solutions in `2_hierarchical_solution.py`. Primitive actions, HTN methods, and the ordered initial task network are worth 1 point each. Solvability with successful independent replay is worth 3 points, averaged across various test configurations. Plan quality contributes the remaining 2 points and is based only on the normalized total utility, for 8 points in total. Test configurations scale up to 10 passengers with a hard 10-second per-case execution limit, as described in the notebook.

Problem 3: Free-form Planning (Optional, 2 Bonus Points)

The optional third task keeps the same passenger-request schema, capacity batching, timing model, and utility function as Problem 2, but removes the required HTN representation. It tests on scenarios with more passengers and uses a hard two-second limit for each case. You may use any finite algorithm to choose the service order; passengers do not walk or change queues.

Instructions: Complete `3_freeform_planning.ipynb`. You will implement the code solutions in the corresponding python script `3_freeform_planning_solution.py`. The bonus is worth at most 2 points. Bonus points earned will fill in any points lost in Problems 1 and 2, with the final total assignment grade capped at 10 points. Twenty equally weighted hidden test cases are used, and each case earns 0.1 point when a valid order attains at least 98% of the optimal total utility within the hard two-second limit. Faster accepted solutions do not receive extra credit.

Through these exercises, you will not only deepen your understanding of AI planning but also gain practical experience in applying these techniques to solve real-world problems.

Good luck, and have fun exploring the fascinating world of AI planning!

Problems You May Face

1. Edge-case of `OneshotPlanner`:

- When executing code blocks that use `OneshotPlanner` from the Unified Planning library, an error may occur if the current file's parent directories contain whitespace in their names.
- Please rename the directories to remove spaces and re-run the code.

2. Packages that are necessary:

- Please install the required packages before running the code:

```
- pip install -r requirements.txt
```